

Day 2 SDE-2 Interview Guide: Bean Scopes, Concurrency & Circular Dependencies

Target Roles: SDE-2 / Senior Backend | Project: Digital Wallet & Payment Ledger
Track: Core Spring Internals (Scopes & 3-Level Cache) & Project Concurrency Drill

Day 2 — Spring Bean Scopes, Concurrency & Circular Dependencies

Think of Day 2 as **two big parts**:

PART 1

Spring Internals

|

- └─ Bean Scopes
- └─ Circular Dependency
- └─ 3-Level Cache
- └─ Prototype + Singleton
- └─ Bean Lifecycle / BPP

PART 2

Concurrency

|

- └─ Singleton + multiple requests
- └─ Shared mutable state
- └─ Race condition
- └─ Stateless service

Q1. What are Bean Scopes in Spring?

First understand what **scope** means.

Suppose we have:

```
@Service
```

```
public class WalletService {  
}
```

Spring creates this object for us.

The question is:

How many objects of WalletService should Spring create?

That's what **Bean Scope** decides.

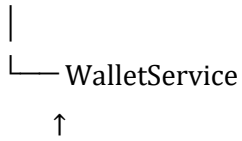
1. Singleton — Default

@Service

```
public class WalletService {  
}
```

By default:

Spring ApplicationContext



ONE OBJECT

There is normally **one WalletService object per Spring ApplicationContext**.

If 1,000 users call it:

User 1 —┐

User 2 —┐

User 3 —┐ —> SAME WalletService object

User 4 —┐

User 5 —┐

This is why we need to be careful with mutable fields.

Easy interview answer

Singleton is the default Spring scope. Spring creates one instance of the bean per ApplicationContext and reuses that instance throughout the application's lifecycle.

2. Prototype

Prototype means:

"Give me a new object whenever Spring is asked for this bean."

@Component

@Scope("prototype")

```
public class Calculator {  
}
```

Conceptually:

getBean(Calculator)



Calculator Object 1

getBean(Calculator)



Calculator Object 2

getBean(Calculator)

↓

Calculator Object 3

Different object every time.

Important:

Spring does **not** manage the complete destruction lifecycle of prototype objects.

3. Request Scope

One object for **one HTTP request**.

Request 1 → Object A

Request 2 → Object B

Request 3 → Object C

Example:

```
@RequestScope
```

```
@Component
```

```
public class RequestData {  
}  
}
```

Useful when you genuinely need data belonging to one HTTP request.

4. Session Scope

One object for one HTTP session.

User A Session → Object A

User B Session → Object B

The same user's object can survive across multiple HTTP requests during that session.

5. Application Scope

Associated with the ServletContext lifecycle.

You can think of it as:

One object associated with the web application's ServletContext.

6. WebSocket Scope

One object for one WebSocket session.

Remember Q1 like this

Singleton → One per ApplicationContext

Prototype → New object when requested

Request → One per HTTP request

Session → One per HTTP session

Application → One per ServletContext/application

WebSocket → One per WebSocket session

Q2. What is Circular Dependency?

This one sounds scary but is actually very simple.

Suppose:

@Service

```
class WalletService {
```

```
    private PaymentService paymentService;
```

```
}
```

Wallet needs Payment.

But Payment also needs Wallet:

@Service

```
class PaymentService {
```

```
    private WalletService walletService;
```

```
}
```

So:

WalletService

↓

needs PaymentService

↓

PaymentService

↓

needs WalletService

↓

WalletService

↓

needs PaymentService

↓

∞

This is called a **circular dependency**.

How does Spring solve it?

This is where the famous **3-level cache** comes in.

Don't think of it as three complicated caches.

Think:

Spring has three places where it can keep track of a singleton while creating it.

Level 1 — Fully Ready Bean

singletonObjects

This means:

"The bean is completely ready."

For example:

WalletService

↓

dependencies injected

↓

@PostConstruct completed

↓

proxy applied

↓

READY

It goes into Level 1.

Level 2 — Early Bean

earlySingletonObjects

This means:

"I have created the object, but it isn't completely initialized yet."

For example:

WalletService object created

↓

dependencies not completely finished

↓

but object reference exists

Spring can temporarily expose that early reference to break certain circular dependencies.

Level 3 — Object Factory

singletonFactories

This is basically:

"If somebody needs this bean early, I know how to produce its early reference."

You don't need to memorize ObjectFactory implementation details right now.

Just remember:

Level 3 = "I know how to give you an early reference."

Level 2 = "Here is the early reference."

Level 1 = "Here is the completely ready bean."

A → B → A Example

Imagine:

A needs B

B needs A

Spring starts:

Create A

↓

A isn't completely ready

↓

Spring keeps a way to expose A early

↓

A needs B

↓

Create B

↓

B needs A

Spring says:

"A isn't ready yet, but I already have an early reference for A."

So B gets that early A.

Then:

B completes

↓

B becomes ready

↓

A gets B

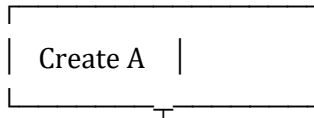
↓

A completes

↓

A becomes ready

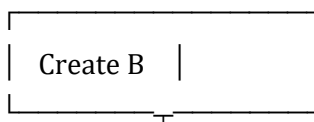
Simple picture:



↓

A needs B

↓



↓

B needs A

↓

"A is already being created"

↓

Give early A reference

↓

B completes

↓

A completes

Interview answer

Spring can resolve certain circular dependencies, especially with setter or field injection, by exposing an early reference to a singleton bean before its complete initialization. Spring internally uses three levels of singleton storage to manage fully initialized beans, early references, and factories capable of producing early references.

Q3. Why doesn't Constructor Injection solve circular dependencies?

This is actually easy.

Consider:

```
class A {
```

```
    A(B b) {
```

```
    }
```

```
}
```

To create A, we **must already have B**.

So:

Create A

↓

Need B

↓

Create B

↓

Need A

↓

Need A to create B

↓

But A needs B

↓



Neither object can even be created.

That's the important difference.

Constructor injection

```
new A(B)
```

B must exist **before A can exist**.

Spring can't say:

"Here is an early A."

because A hasn't even been constructed yet.

Interview answer

Constructor injection creates a dependency before the bean itself can be instantiated. Therefore, if A requires B in its constructor and B requires A in its constructor, neither object can be created first, so Spring cannot resolve the circular dependency and typically fails with `BeanCurrentlyInCreationException`.

Q4. Prototype Bean Injected into Singleton

This is one of the most important questions.

Suppose:

```
@Component
```

```
@Scope("prototype")
```

```
public class Calculator {
```

```
}
```

And:

```
@Service
```

```
public class WalletService {
```

```
    @Autowired
```

```
    private Calculator calculator;
```

```
}
```

You might think:

"Calculator is prototype, so every WalletService method call will get a new Calculator."

No.

Why?

Because WalletService itself is a singleton.

Spring creates:

WalletService

↓

during creation

↓

gets Calculator

↓

Calculator Object #1

Now:

WalletService

|

↓

Calculator #1

If you call:

```
walletService.transfer();
```

```
walletService.transfer();
```

```
walletService.transfer();
```

you still have:

WalletService

|

↓

Calculator #1

It doesn't automatically ask Spring for a new prototype every time.

How do we get a fresh prototype?

Solution 1 — ObjectProvider

```
private final ObjectProvider<Calculator> calculatorProvider;
```

Then:

```
Calculator calculator =
```

```
    calculatorProvider.getObject();
```

Now you're saying:

"Spring, give me a new Calculator right now."

Every time you call it, Spring can provide a fresh prototype instance.

```
transfer()
```

↓

```
getObject()
```

↓

```
Calculator #1
```

```
transfer()
```

↓

```
getObject()
```

↓

```
Calculator #2
```

```
transfer()
```

↓

```
getObject()
```

↓

```
Calculator #3
```

Solution 2 — Scoped Proxy

You can ask Spring to inject a proxy instead.

Conceptually:

```
WalletService
```

↓

```
Calculator Proxy
```

↓

```
actual prototype Calculator
```

The proxy asks Spring for the appropriate prototype target when the bean is used.

But remember our TransferFeeCalculator!

We **don't need prototype** for our fee calculator.

We fixed it by making it:

Singleton

+

Stateless

=

Thread-safe

That's generally much simpler.

Q5. Does Spring call @PreDestroy for Prototype?

No.

For singleton:

Spring creates object

↓

Spring manages it

↓

Application shuts down

↓

Spring calls @PreDestroy

For prototype:

Spring creates object

↓

gives it to you

↓

Spring doesn't fully manage its destruction

So:

@PreDestroy

```
public void cleanup() {
```

```
}
```

is **not automatically called by Spring** for prototype instances.

Easy interview answer

No. Spring manages creation and configuration of prototype beans but does not manage their destruction lifecycle. Therefore, @PreDestroy is not automatically invoked for prototype instances. The client is responsible for cleanup of resources owned by the prototype.

PART 2 — Concurrency

Now we come to the part we actually practiced.

Q6. Why is mutable state in a Singleton Service dangerous?

Our bad code was:

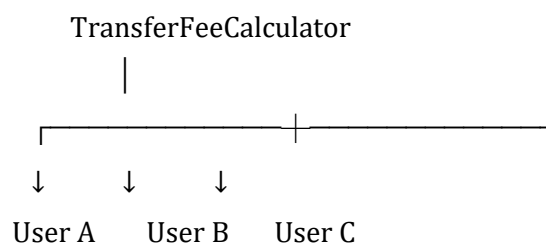
```
@Service
```

```
public class TransferFeeCalculator {
```

```
    private Money lastCalculatedFee;
```

```
}
```

Spring creates one object:



All users are using **the same object**.

And this:

```
private Money lastCalculatedFee;
```

is shared.

So:

User A → writes \$50

User B → writes \$0.10

User C → writes \$20

They're all modifying the same field.

That's the problem.

Why does this happen?

Because web applications handle requests concurrently.

Conceptually:

Request A → Thread A

Request B → Thread B

Request C → Thread C

And:

Thread A —┐

Thread B —┴—> SAME Service object

Thread C —┐

Therefore:

Shared object

+

Mutable field

+

Multiple threads

=

Potential race condition

Q7. What happened in our test?

We had:

User 1 → \$5,000 → fee = \$50

User 2 → \$100 → fee = \$0.10

Both used the same calculator.

Eventually the shared field contained:

lastCalculatedFee = \$50

So:

calculator.getLastCalculatedFee()

returned:

\$50

The important point is:

The final field belongs to the shared singleton, not to a particular user.

And that's exactly why storing request-specific information in service fields is dangerous.

Q8. How does Stateless Architecture solve this?

This is the **most important lesson from our coding exercise.**

Bad:

```
private Money lastCalculatedFee;
```

Good:

```
public Money calculateFee(Money transferAmount) {
```

```
BigDecimal feeValue = ...;
```

```
return new Money(  
    feeValue,  
    transferAmount.currency()  
);  
}
```

Now:

User A

↓

```
calculateFee($5000)
```

↓

```
feeValue = $50
```

↓

```
return $50
```

At the same time:

User B

↓

```
calculateFee($100)
```

↓

```
feeValue = $0.10
```

↓

```
return $0.10
```

They don't overwrite each other.

Why?

Because the method's local variables belong to that particular method execution.

Think:

Thread A

feeValue = \$50

Thread B

feeValue = \$0.10

Different method calls → different local execution state.

Interview answer

I keep the Spring service as a singleton but make it stateless. Request-specific data is passed through method parameters and local variables instead of mutable instance fields. Therefore, concurrent requests don't modify shared request state, eliminating this class of race condition without synchronization locks.

Q9. Why do we need BeanPostProcessor?

Now this should be easy after our discussion.

Suppose we have 100 services.

We want to automatically:

log beans

measure initialization

validate beans

instrument beans

wrap beans

We don't want to modify every service.

So:

Spring creates bean

↓

BeanPostProcessor

↓

inspect / modify / wrap

↓

bean continues

Our BPP:

postProcessBeforeInitialization()

means:

"Let me see this bean before initialization finishes."

And:

postProcessAfterInitialization()

means:

"Let me see this bean after initialization."

Our current BPP simply logs:

BEFORE INIT

↓

initialization

↓

AFTER INIT

But a BPP can do more than logging. It can participate in framework infrastructure that wraps beans with proxies.

Interview answer

A BeanPostProcessor is a Spring extension point that allows us to inspect, modify, or wrap beans before and after initialization. It is useful for application-wide logging, instrumentation, validation, and proxy-based behavior without modifying individual service classes.

Q10. Why prefer Stateless Singleton for high-throughput services?

This question has an important idea but your original answer is **a little too aggressive about GC.**

Don't say:

"Singleton is always faster because prototype creates GC pauses."

That's not always true.

The real reason is simpler:

Stateless singleton

Create service once

↓

Reuse it

↓

Many requests can use it

↓

No request-specific mutable state

Very efficient.

Prototype

Request/use

↓

new object

↓

another object

↓

another object

More object creation can mean more allocation and potentially more GC pressure.

But **don't choose singleton merely because of GC.**

Choose it when the service can safely be stateless.

The Golden Rule for Your Interview

Remember this:

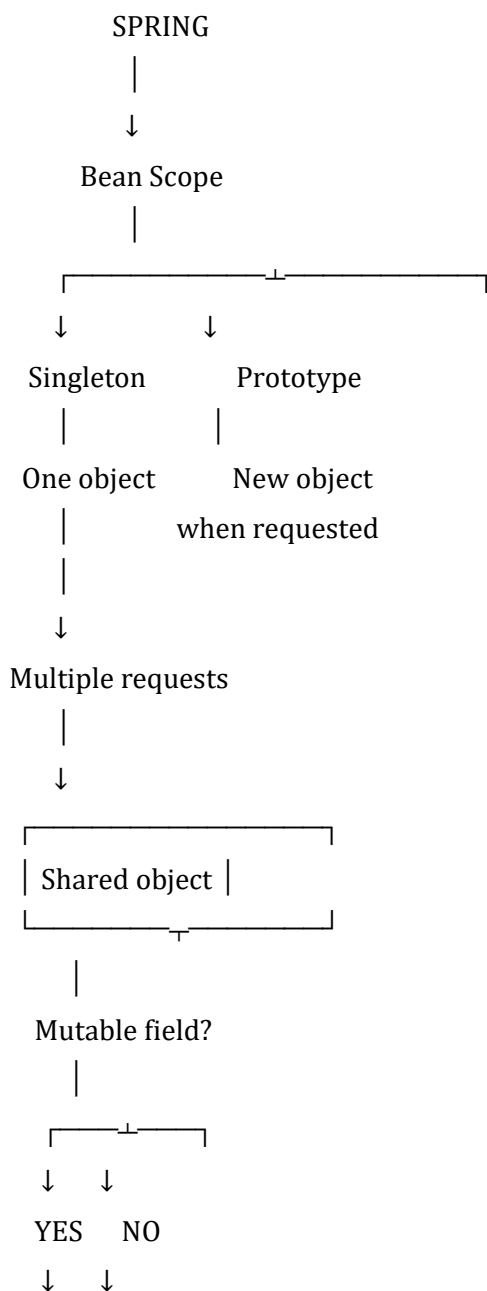
If a service doesn't need to maintain per-request/per-user mutable state, keep it as a singleton and make it stateless.

That's the architecture we used for:

TransferFeeCalculator

🔥 Entire Day 2 in One Picture

This is what I want you to remember instead of memorizing 10 long answers:



Problem Safe

|

↓

Make service

STATELESS

|

↓

Parameters + Local Variables

|

↓

Thread-safe

And separately:

Spring Bean Creation

↓

BeanPostProcessor

↓

BEFORE initialization

↓

@PostConstruct / initialization

↓

AFTER initialization

↓

Possible proxy/wrapping

↓

Bean ready

And:

$A \rightarrow B \rightarrow A$

↓

Circular Dependency

↓

Setter/Field injection

↓

Spring can sometimes expose
an early singleton reference

↓

3-Level Singleton Cache

But:

A constructor needs B

B constructor needs A

↓

Neither can be created first

↓

Circular dependency failure

Final Day 2 Cheat Sheet

Question	Remember this
Bean Scope	Controls how long/how many bean instances Spring creates
Singleton	One instance per ApplicationContext
Prototype	New instance when requested
Request	One instance per HTTP request
Session	One instance per HTTP session
Circular Dependency	A needs B, B needs A
3-Level Cache	Helps Spring expose early singleton references
Constructor Circular Dependency	Can't resolve because objects aren't constructible yet
Prototype → Singleton	Injection happens when singleton is created, so same injected prototype is reused
ObjectProvider	Ask Spring for a fresh prototype when needed
@PreDestroy + Prototype	Not automatically called
Mutable Singleton	Shared by concurrent requests → race-condition risk
Stateless Service	No request-specific mutable instance fields
BPP	Hook to inspect/modify/wrap beans during initialization
High-throughput service	Prefer stateless singleton when possible

One final interview-level answer tying the whole Day 2 together:

"In Spring, services are singleton-scoped by default, so the same service instance can handle many concurrent requests. Therefore, I avoid storing request-specific mutable state in instance fields and instead design services to be stateless. For cases involving bean lifecycle customization, Spring provides BeanPostProcessor, which allows us to inspect, modify, or wrap beans during

initialization. Spring can also resolve certain circular dependencies through early singleton references, although constructor-based circular dependencies cannot be resolved because neither object can be instantiated first."